



LIFE OS

Engineering Handbook

A personal iOS app that unifies every signal from the devices you already wear and carry — Apple Watch, iPhone, Whoop, Renpho — and reasons over all of it to say what's working and what isn't. This handbook is the single reference for how it is built, why it is built that way, and what it does when things go wrong.

REVISION B 10 AUGUST 2026 IOS 26 · SWIFT 6 ARCHITECTURE: MVVM STATUS: V1 IN BUILD
AUDIENCE: PERSONAL

Table of contents

Every line is a link as well as a page number. Chapters 4 to 9 are the features, one each.

1	Architecture	3	8	Settings	29
1.1	System context	3	8.1	Intent	29
1.2	Module map	3	8.2	Files and folders	29
1.3	The DailyMetrics spine	4	8.3	Model	30
1.4	Local first topology	5	8.4	ViewModel	30
1.5	Conflict resolution	6	8.5	View	31
2	MVVM and project structure	7	8.6	Edge cases and tests	31
2.1	Why MVVM here	7	9	The Coach	32
2.2	What each layer owns	8	9.1	Intent	32
2.3	The screen first directory layout	9	9.2	Files and folders	32
2.4	Anatomy of a feature folder	10	9.3	Model	33
2.5	Shared components	11	9.4	ViewModel	34
2.6	Rules and review checklist	12	9.5	View	34
3	Design system	13	9.6	Edge cases and tests	34
3.1	Principle	13	10	Integrations	36
3.2	Tokens	13	10.1	Apple HealthKit	36
3.3	Component inventory	14	10.2	Whoop	37
3.4	Dark mode	14	10.3	Supabase	38
4	Today	15	10.4	Foundation Models	38
4.1	Intent	15	10.5	Screen Time	39
4.2	Files and folders	16	11	Data reference	40
4.3	Model	16	11.1	DailyMetrics	40
4.4	ViewModel	17	11.2	UserGoals	40
4.5	View	18	11.3	Source records	41
4.6	Edge cases and tests	18	11.4	Goal evaluation	41
5	Body	20	11.5	Store protocols	41
5.1	Intent	20	12	Production edge cases	43
5.2	Files and folders	20	12.1	Time and calendars	43
5.3	Model	21	12.2	HealthKit failures	43
5.4	ViewModel	21	12.3	Whoop failures	44
5.5	View	22	12.4	Sync failures	44
5.6	Edge cases and tests	22	12.5	UI and data display	45
6	Activity	23	12.6	AI failures	45
6.1	Intent	23	13	Operations	46
6.2	Files and folders	23	13.1	Setup runbook	46
6.3	Model	24	13.2	Secrets	47
6.4	ViewModel	24	13.3	Testing strategy	47
6.5	View	24	A	Product thesis	49
6.6	Edge cases and tests	25	A.1	The thesis	49
7	Recovery	26	A.2	Product PR FAQ	50
7.1	Intent	26	A.3	Roadmap	50
7.2	Files and folders	26	B	Decision log	52
7.3	Model	27			
7.4	ViewModel	27			
7.5	View	28			
7.6	Edge cases and tests	28			

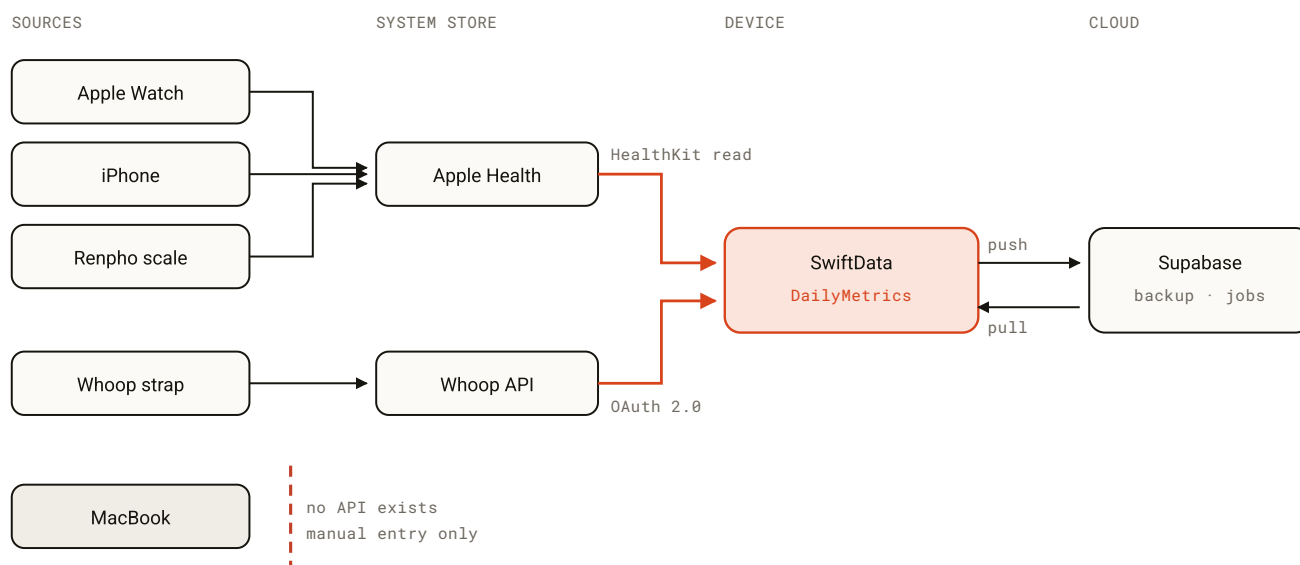
1 Architecture

Life OS is a local-first iOS app built on one bet: **the value is in the join, not the collection.** Every architectural decision below follows from it. Four devices produce data, all of it converges on a single table keyed by calendar day, and every screen reads from that table and nothing else.

This document describes a system that is designed but not finished — V1 is in active implementation. Where the code and this handbook disagree, the code is right and this handbook is a bug. The product reasoning that led here is in Appendix A; it is deliberately behind the engineering, not in front of it.

1.1 System context

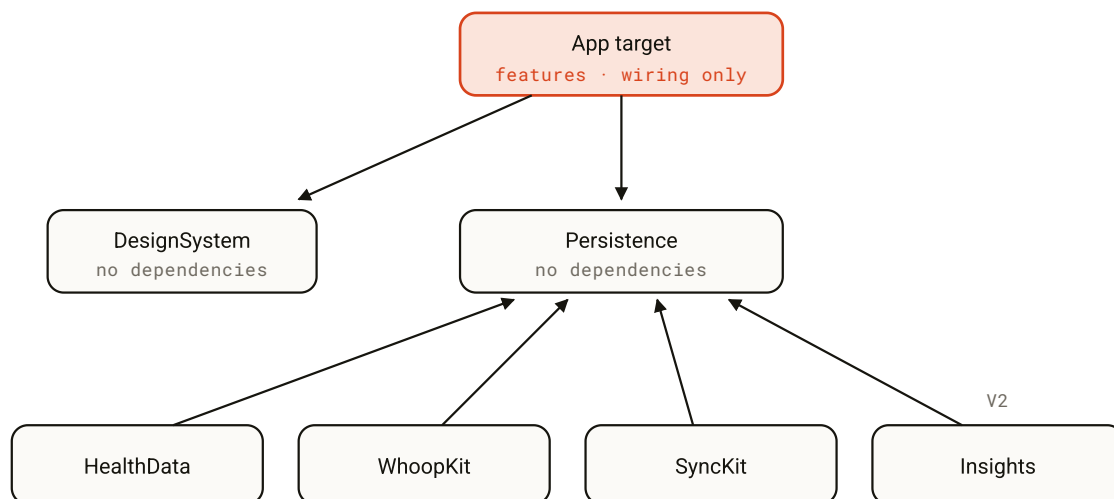
Four devices produce data, two of them reach the app through Apple Health, one through a REST API, and one — your MacBook — cannot reach it at all. Everything converges on a single local store before anything renders.



Every automatic source converges on one local table before anything renders. The MacBook has no programmatic path at any privilege level — that is a platform fact, not a missing feature.

1.2 Module map

`LifeOSKit` is a local Swift package providing compile-time boundaries. The app target contains only features and wiring, laid out per screen as described in Chapter 2. The dependency rules are enforced by the compiler, not by convention.



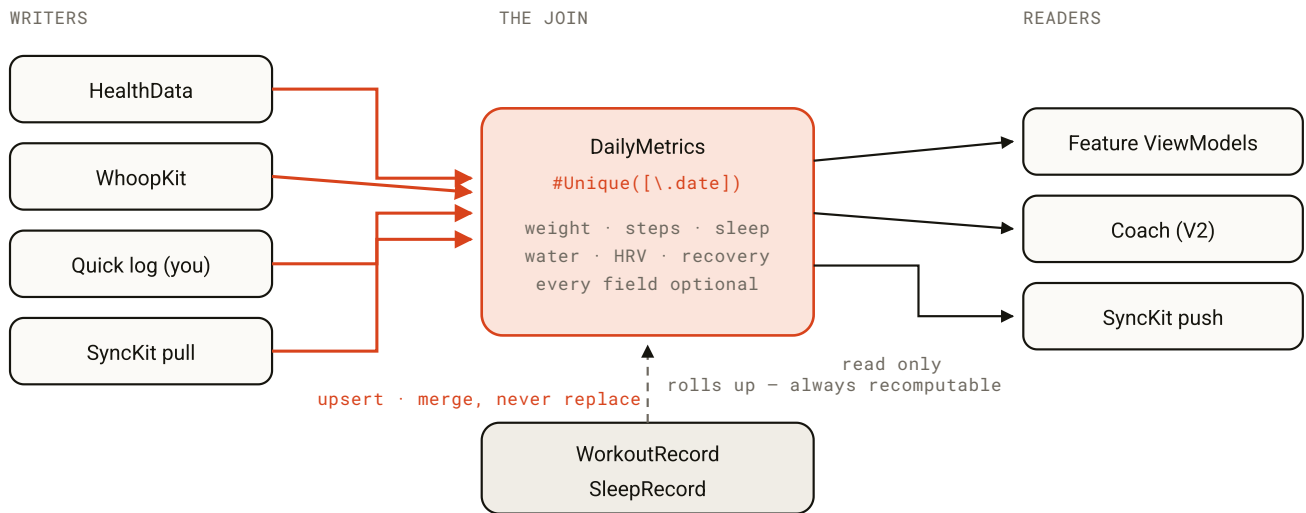
Arrows read "depends on". No arrow between siblings – that is the point.

The four integration modules are siblings that cannot see each other. Rewriting the Whoop client cannot touch a view, and `Insights` reads only from `Persistence` – it never calls HealthKit or Whoop directly.

RULE	ENFORCED HOW	WHY
Nothing outside <code>HealthData</code> imports HealthKit	Package dependency graph	HealthKit types never leak into views; the module is swappable and testable
Nothing outside <code>WhoopKit</code> knows Whoop exists	Package dependency graph	A vendor change is contained to one target
Nothing outside <code>SyncKit</code> imports the Supabase SDK	Package dependency graph	The app works identically with sync disabled
<code>Insights</code> reads <code>Persistence</code> only	Package dependency graph	The coach cannot make network calls mid-conversation
<code>DesignSystem</code> depends on nothing	Package dependency graph	Components stay previewable and reusable across every module
No feature folder imports another feature folder	Review; see 2.6	Screens compose only through <code>Components</code> and <code>DesignSystem</code>

1.3 The DailyMetrics spine

This is the single most consequential decision in the system. **One row per calendar day.** Every source writes into it; every reader reads from it.



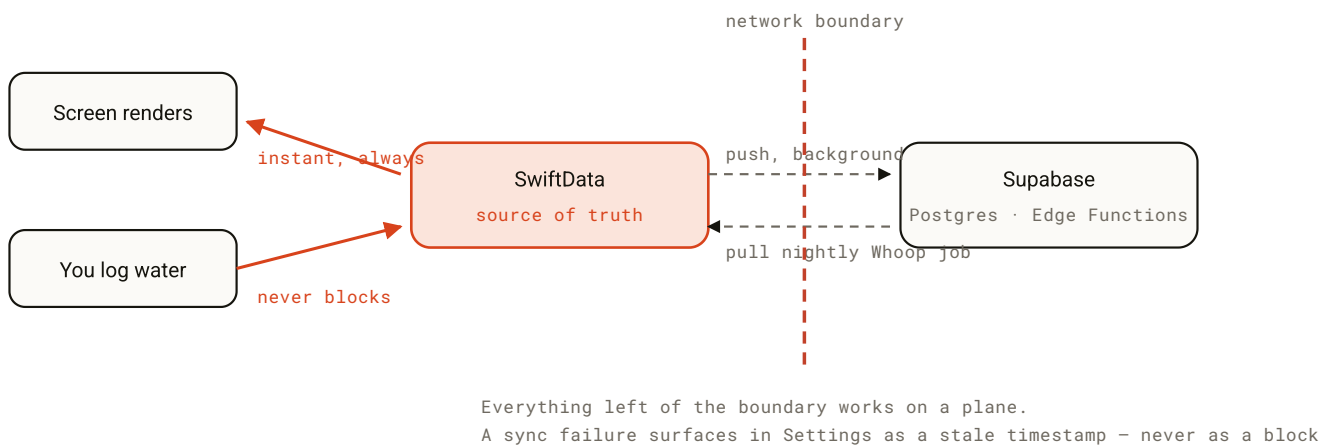
Sources merge into the row rather than replacing it, so HealthKit and Whoop writing the same day never clobber each other. `DailyMetrics` is derived state: it can always be rebuilt from the source records beneath it.

WHAT THE SINGLE-TABLE CHOICE BUYS

- **Trends become a sort**, not a multi-source asynchronous join.
- **The coach queries one table** instead of orchestrating two network APIs mid-conversation — which is what makes a 3B on-device model viable.
- **Offline works by default**, because nothing renders from a live network call.
- **Adding a source later** means writing into the same row, not building a new pipeline.
- **Every ViewModel has the same dependency** — one store protocol — so screens stay uniform and stubbable.

1.4 Local first topology

SwiftData on-device is the source of truth for the UI. Supabase is backup, sync, and a place to run jobs that a sleeping phone cannot.



Reads and writes both terminate locally. The dashed sync paths are the only network-dependent edges in the entire app, and neither is on a user-facing critical path.

1.5 Conflict resolution

Field-level last-write-wins, using `updatedAt` . Not row-level.

Row-level LWW would be a bug here. The nightly Whoop job writes recovery fields at 04:00; if you logged water at 23:00 the night before, a row-level overwrite silently discards it. Field-level merge means each writer touches only its own columns.

This is defensible only because there is exactly one human user and the writers rarely contest the same field: HealthKit owns the health columns, the Whoop job owns the Whoop columns, and you own the manual ones. If this app ever became multi-user, this strategy would need replacing outright.

KNOWN LIMITATION

LWW depends on clock agreement between the device and the Edge Function. Device clock skew of more than a few minutes can cause a stale write to win. Accepted for a single-user personal app; documented so it isn't rediscovered as a mystery bug.

2 MVVM and project structure

Every screen in Life OS is built the same way: a Model of value types, a ViewModel that owns all state and every decision, and a View that renders and forwards intent. The folder layout mirrors that exactly, one folder per screen, so a file's path tells you its layer before you open it.

2.1 Why MVVM here

SwiftUI will happily let a view query the database. `@Query` in a `View` is one line and works immediately — and it is the reason this app does not do it. Three specific things break:

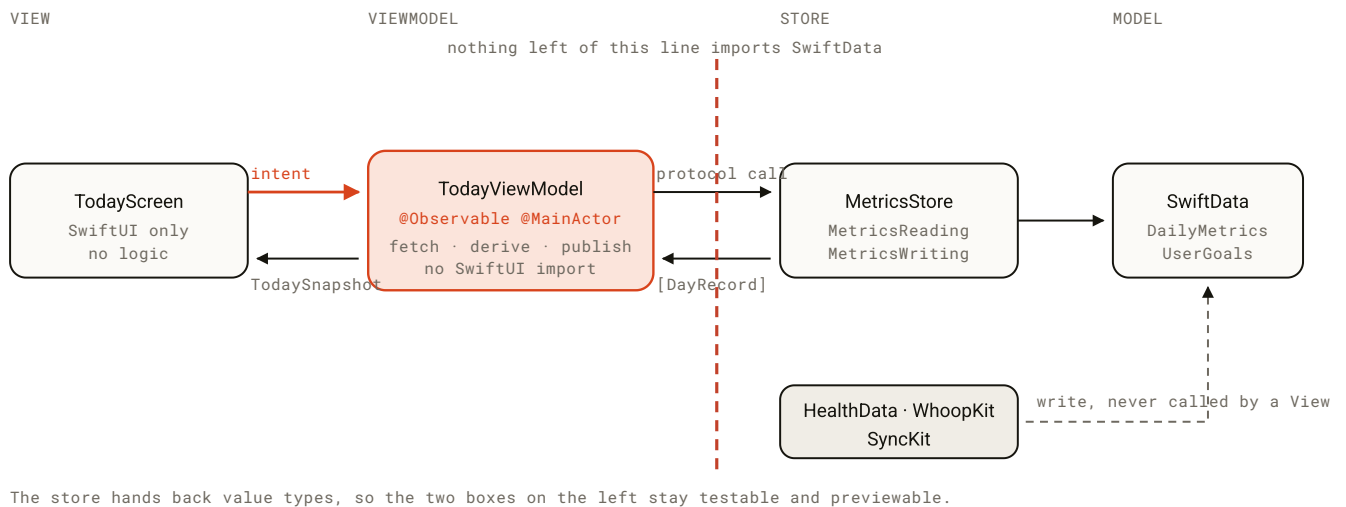
- **Nothing is testable without a simulator.** Goal evaluation, streak arithmetic, and weekly deltas are the parts most likely to be wrong, and they would live inside a view body that only a running device can exercise.
- **Derivation gets re-run per render.** Status-per-day, the month grid, the streak — computed in `body`, they recompute on every state change, including ones that have nothing to do with them.
- **Persistence types leak into layout code.** Once a view knows what a `@Model` class is, changing the schema means editing views.

MVVM fixes all three for the price of one extra file per screen. The ViewModel does the fetching and all derivation once, hands the View a plain value type, and can be exercised in a unit test with a stub store in milliseconds.

THE RULE THAT MAKES IT WORK

A ViewModel never imports SwiftUI, and a View never imports SwiftData. Both are checkable by reading the first ten lines of a file, and between them they force everything else into place.

2.2 What each layer owns



The View sends intent and renders a snapshot. The ViewModel is the only place that decides anything. The store is a protocol, which is what lets a test replace it with four lines of stub.

LAYER	LIVES IN	SHAPE	MAY IMPORT
Domain model	LifeOSKit/Sources/Persistence	<code>@Model</code> classes, store, pure logic	Foundation, SwiftData
Feature model	Features/<Screen>/Model	<code>struct</code> , <code>Sendable</code> , <code>Equatable</code> — the screen's rendered state	Foundation, Persistence value types
ViewModel	Features/<Screen>/ViewModel	<code>@Observable</code> , <code>@MainActor</code> final class	Foundation, Observation, Persistence — never SwiftUI
View	Features/<Screen>/View	<code>struct: View</code> , plus its private subviews	SwiftUI, DesignSystem, its own ViewModel — never SwiftData
Shared component	Components/{Model, ViewModel, View}	Same three shapes, reused by two or more screens	As above
Design primitive	LifeOSKit/Sources/DesignSystem	Stateless <code>View</code> structs and tokens	SwiftUI only — knows nothing about the app

The distinction between the last two rows is the one people get wrong. `DesignSystem` holds things that would make sense in any app — `HeroNumeral`, `DotGrid`, `GlassCard`. `Components` holds things that are reusable but specifically about *this* app's data — the quick-log sheet, the connection row, the metric tile that knows a missing value renders as an em dash.

2.3 The screen first directory layout

Folders are named after screens, not after layers. `Features/Body/` holds everything the Body screen is, and inside it the three MVVM folders classify each file. Finding code starts with "which screen," which is how anyone actually thinks about a bug report.

```
LifeOS/                                app target - file-system synchronised
├── App/
│   ├── LifeOSApp.swift                @main, container + store injection
│   ├── RootView.swift                tab bar, FAB, screen hosting
│   └── RootViewModel.swift            selected tab, FAB menu, quick-log presentation
├── Features/                          one folder per screen
│   ├── Today/
│   │   ├── Model/
│   │   │   └── TodaySnapshot.swift
│   │   ├── ViewModel/
│   │   │   └── TodayViewModel.swift
│   │   └── View/
│   │       ├── TodayScreen.swift
│   │       ├── TodayHeader.swift
│   │       └── TodayTileGrid.swift
│   ├── Body/
│   │   ├── Model/BodySnapshot.swift
│   │   ├── ViewModel/BodyViewModel.swift
│   │   └── View/BodyScreen.swift, WeightBarChart.swift
│   ├── Activity/
│   │   ├── Model/ActivitySnapshot.swift
│   │   ├── ViewModel/ActivityViewModel.swift
│   │   └── View/ActivityScreen.swift
│   ├── Recovery/
│   │   ├── Model/RecoverySnapshot.swift
│   │   ├── ViewModel/RecoveryViewModel.swift
│   │   └── View/RecoveryScreen.swift
│   ├── Settings/
│   │   ├── Model/SettingsSnapshot.swift, ConnectionState.swift
│   │   ├── ViewModel/SettingsViewModel.swift
│   │   └── View/SettingsScreen.swift, GoalEditorSection.swift
│   └── Coach/                          V2
│       ├── Model/CoachBrief.swift, ChatTurn.swift
│       ├── ViewModel/CoachViewModel.swift
│       └── View/CoachScreen.swift, BriefCard.swift
├── Components/                        shared across two or more screens
│   ├── Model/
│   │   ├── MetricReading.swift        value + goal + unit, or nothing
│   │   └── ConnectionStatus.swift
│   ├── ViewModel/
│   │   ├── QuickLogViewModel.swift
│   │   └── ConnectionRowViewModel.swift
│   └── View/
│       ├── QuickLogSheet.swift
│       ├── MetricTile.swift
│       ├── MetricValueLabel.swift     the em-dash rule, in one place
│       └── ConnectionRow.swift
```

```
└─ Resources/
  └─ Assets.xcassets
```

```
LifeOSKit/                                local Swift package – no UI state, no screens
└─ Sources/
  └─ DesignSystem/                        tokens + stateless primitives
  └─ Persistence/                        @Model types, store, pure logic
```

WHY THIS WORKS WITHOUT TOUCHING THE PROJECT FILE

The app target uses a `PBXFileSystemSynchronizedRootGroup`. Any folder created under `LIfeOS/` is picked up by Xcode automatically – the directory structure *is* the project structure, and no one has to remember to add files to a target. Moving a screen between folders is a `git mv`, with no project file to merge.

THE COROLLARY: DON'T PUT DOCUMENTATION IN THESE FOLDERS

Synchronised membership means *every* file is a build input. A `README.md` in each folder is copied into the app bundle as a resource, and twenty-three of them collide on the same destination path – Multiple commands produce `.../LIfeOS.app/README.md`. Empty folders are held open by a hidden `.gitkeep`, which Xcode ignores; the layer contracts are documented here instead.

2.4 Anatomy of a feature folder

Every feature folder answers the same four questions in the same order. Chapters 4 to 9 are each one instance of this template.

FOLDER	CONTAINS	TEST
<code>Model/</code>	One snapshot struct per screen, plus any small enums it needs. Value types only. No methods that fetch, no dates computed from <code>.now</code> .	Compiles without SwiftUI or SwiftData
<code>ViewModel/</code>	Exactly one class. Holds <code>private(set) var snapshot</code> , a <code>loadState</code> , the store dependency, and the <code>func</code> the view calls.	Instantiable in a unit test with a stub store
<code>View/</code>	The screen struct and its private subviews. Reads only from its view model; owns <code>@State</code> for presentation concerns like sheet visibility and focus.	Every subview has a <code>#Preview</code> in both schemes

The ViewModel shape is identical everywhere, which is the point:

```

@Observable
@MainActor
final class BodyViewModel {
    private(set) var snapshot: BodySnapshot = .empty
    private(set) var state: LoadState = .idle

    private let store: MetricsReading
    private let calendar: Calendar

    init(store: MetricsReading, calendar: Calendar = .current) {
        self.store = store
        self.calendar = calendar
    }

    func load() async { ... }           // called from .task
    func refresh() async { ... }        // called from .refreshable
}

```

`LoadState` is a four-case enum shared by every screen — `.idle`, `.loading`, `.loaded`, `.failed(String)` — and lives in `Components/Model`. One vocabulary for "what is this screen doing right now" is worth more than per-screen nuance.

2.5 Shared components

`Components/` is split into the same three folders, for the same reason: a shared piece of UI that owns state has a view model, and it belongs somewhere obvious.

COMPONENT	MODEL	VIEWMODEL	USED BY
<code>MetricTile</code>	<code>MetricReading<Value></code>	None — stateless	Today, Activity, Recovery, Body
<code>MetricValueLabel</code>	<code>MetricReading<Value></code>	None — stateless	Every screen with a number on it
<code>QuickLogSheet</code>	<code>QuickLogDraft</code>	<code>QuickLogViewModel</code> — writes water and weight	Root FAB, from any tab
<code>ConnectionRow</code>	<code>ConnectionStatus</code>	<code>ConnectionRowViewModel</code> — reconnect actions	Settings; Recovery's empty state

A component earns its place in `Components/` the second time it is needed, not the first. The first time, it is a private subview inside the feature that needs it. Promoting it is a file move between two folders that already exist.

WHERE THE EM-DASH RULE LIVES

"A missing metric renders as an em dash at 35% opacity, never as `0`" is the single most repeated rule in this app. It is implemented once, in `Components/View/MetricValueLabel.swift`, and every screen renders numbers through it. A rule stated in a handbook and implemented in nine places is a rule that will be broken in one of them.

2.6 Rules and review checklist

RULE	ENFORCED HOW	WHY
A View never imports <code>SwiftData</code>	Review – visible in the imports	All reads go through a <code>ViewModel</code> ; the schema can change without touching layout
A <code>ViewModel</code> never imports <code>SwiftUI</code>	Review – visible in the imports	Keeps it unit-testable and prevents <code>View</code> -shaped thinking leaking in
<code>ViewModels</code> depend on <code>MetricsReading</code> , not <code>MetricsStore</code>	Compiler – the init takes the protocol	A test injects a stub; a preview injects seed data
The store returns value types, never <code>@Model</code> instances	Compiler – protocol signatures return <code>DayRecord</code>	Nothing above the store can accidentally mutate the database
One <code>ViewModel</code> per screen, constructed by the View that owns it	Review	<code>@State private var vm = ...</code> ties lifetime to the screen
No feature folder imports another feature folder	Review	Sharing goes through <code>Components</code> , which forces the shared thing to be named
Derivation happens in the <code>ViewModel</code> , never in <code>body</code>	Review	<code>body</code> runs far more often than data changes
Snapshot types are <code>Equatable</code>	Compiler	Cheap re-render skipping, and it makes <code>ViewModel</code> tests one-liners

WHAT IS SCAFFOLDED AND WHAT IS NOT, AS OF REVISION B

The folder tree above exists and the five V1 screens sit in their `View` folders. **No `ViewModel` is written yet**, so the `Model` and `ViewModel` folders are empty, and the store protocols in 11.5 do not exist either – the screens still reach `MetricsStore` through `@Query` in the view body, which is what the V1 plan (Tasks 9–12) specified before this chapter existed. Moving each query into a `ViewModel` is the outstanding work; nothing else in the plan changes.

3 Design system

Ten modules have to read as one app. That happens because they share a small set of primitives and differ by exactly one variable – the module hue – rather than because each screen was designed carefully on its own.

3.1 Principle

The hub is neutral; the domains are coloured. Today is a warm off-white canvas. Every domain screen is a saturated-at-top, near-white-at-bottom gradient in its own hue. This resolves two competing visual languages into one hierarchy instead of a collision, and it means adding a module later is a token, not a design exercise.

3.2 Tokens

TOKEN	VALUE	MEANING
<code>ModuleHue.body</code>	Teal	Weight and composition
<code>ModuleHue.activity</code>	Amber	Movement
<code>ModuleHue.recovery</code>	Blue	Whoop, HRV, sleep quality
<code>ModuleHue.nutrition</code>	Purple	V3
<code>ModuleHue.money</code>	Green	V6
<code>ModuleHue.habits</code>	Orange	V4
<code>LifeOSTokens.accent</code>	Orange, fixed	Exactly two meanings: <i>today</i> , and <i>act</i>
<code>LifeOSTokens.canvas</code>	Warm off-white	The neutral hub

The accent never varies by module. It marks the current day, the FAB, the selected tab, and alert states. Everything else on screen is the module hue or a neutral. Two colours competing for "this is important" is the fastest way to make neither mean anything.

Colours are `AdaptiveColor` pairs resolved explicitly by the consuming view rather than UIKit dynamic colours, because the package also builds for macOS so that `swift test` runs from the terminal without a simulator. Explicit resolution is the price of fast tests.

3.3 Component inventory

PRIMITIVE	WHAT IT IS	NOTES
<code>GradientCanvas(hue:)</code>	The one gradient primitive: saturated top → near-white bottom	Adding a module is a hue token, not a new screen design
<code>GlassCard</code> / <code>SolidCard</code>	Two card treatments by zone — frosted over the saturated region, solid with a soft shadow once the background has gone pale	What makes cards legible at both ends of a gradient
<code>DotGrid</code>	The shared vocabulary of the whole app: filled = target met, hollow = missed, orange = today, grey = future, faint outline = no data	Month view on Today, streaks on Habits (V4), "days on target" inside any domain screen
<code>MonthGridLayout</code>	Pure calendar maths producing the grid's cells	No SwiftUI; fully unit-tested against a fixed UTC calendar
<code>HeroNumeral</code>	~96pt semibold numeral with tight tracking and a quiet label beneath	Never renders a numeral without its unit
<code>StatTile</code>	Small metric tile: label, value, optional progress	Wrapped by <code>Components/View/MetricTile</code> , which adds the app's missing-value rule
<code>WeekStrip</code>	Pinned seven-day header, today filled orange	Selection is a binding — the owning ViewModel decides what selection means
<code>MonthCalendarView</code>	<code>WeekdayHeader</code> , <code>DateHeadline</code> , and the assembled month	Takes <code>[DotCell]</code> and a date — no data access, so a snapshot literal previews it

The tab bar is circular icons plus a floating dark FAB for logging. **In V1 the FAB logs water and weight only** — the two metrics with no automatic source. Both write to local `DailyMetrics` through `QuickLogViewModel`; the app requests no HealthKit write permissions in V1. The FAB's menu grows in later slices (meals in V3, habits in V4).

3.4 Dark mode

Every hue has a dark variant: deep saturated top, near-black bottom, card materials inverted. These are defined as tokens in V1, before any screen exists, because retrofitting a gradient system to dark mode later is significantly harder than doing it once at the start.

NO COLOUR IS DEFINED ONLY INSIDE A SCHEME BRANCH

Every token has a light value and a dark value, always both. A colour that exists only in the dark branch renders as whatever the parent inherited in light mode — usually black on black, and usually discovered by a user rather than a preview.

4 Today

The hub. Neutral canvas, oversized date, week strip, a month of dots, and four tiles. It answers "am I on track" before you have processed a single number.

4.1 Intent

WORKING BACKWARDS · V1 FEATURE

Today opens with the date at the size of a poster, a strip of the current week, and a grid of dots — one per day this month. Filled dots are days you hit your targets, hollow dots are days you didn't, and today is orange. Underneath, four tiles: steps, sleep, weight, recovery. You can read the whole month in about a second.

What makes a dot fill?

Meeting **3 of 4** daily goals: steps $\geq 8,000$, sleep $\geq 7h$, exercise ≥ 30 min, water $\geq 2,500$ ml. All five values — the four goals and the required count — are editable in Settings. The rule is deliberately explicit; a vague definition makes the entire grid meaningless.

What happens on a day with no data at all?

It renders as no-data, not as a miss, and it is **excluded from streak arithmetic entirely**. Leaving your watch on the charger is not a moral failure, and a system that punishes it teaches you to distrust the streak.

And a day with *some* data?

Judged on what it has. If steps are logged at 500 and nothing else was recorded, that's a miss. The alternative — treating partial days as blank — would let forgetting to log water silently launder a bad day into a neutral one.

If I change a goal, does history change?

Yes. Status is computed on read, never stored, so past dots re-evaluate against the new target immediately. The grid answers "how am I doing against my current standard," not "how did I do against a standard I've since abandoned."

SCREEN ANATOMY

- **Header** — day numeral at 92pt, month and year, weekday, all derived from the selected date.
- **WeekStrip** — seven days, today filled orange, selection bound to the ViewModel.
- **Month card** — month label, streak count in accent, `DotGrid`.
- **Tile grid** — steps, sleep, weight, recovery; each a `MetricTile`.

4.2 Files and folders

```
Features/Today/  
├─ Model/  
│   └─ TodaySnapshot.swift          rendered state, no behaviour  
├─ ViewModel/  
│   └─ TodayViewModel.swift        fetch, evaluate, derive grid + streak  
└─ View/  
    ├─ TodayScreen.swift           layout only  
    ├─ TodayHeader.swift           the oversized date  
    └─ TodayTileGrid.swift         the four tiles
```

Consumes from the package: `DotGrid`, `MonthGridLayout`, `WeekStrip`, `SolidCard`, `evaluate`, `currentStreak`. Consumes from `Components`: `MetricTile`.

4.3 Model

```
struct TodaySnapshot: Equatable, Sendable {  
    var selectedDate: Date  
    var monthLabel: String  
    var cells: [DotCell]                // from MonthGridLayout  
    var streak: Int  
    var steps: MetricReading<Int>  
    var sleepMinutes: MetricReading<Int>  
    var weightKg: MetricReading<Double>  
    var recoveryPct: MetricReading<Double>  
  
    static let empty = TodaySnapshot(...)  
}
```

`MetricReading` is the shared component model: a value that may be absent, and a goal that may not exist. It is what carries the missing-versus-zero distinction from the database all the way to the label without any screen having to remember it.

```
struct MetricReading<Value: Equatable & Sendable>: Equatable, Sendable {  
    var value: Value?                  // nil renders an em dash – never 0  
    var goal: Value?  
    var unit: String?  
}
```

4.4 ViewModel

```
@Observable
@MainActor
final class TodayViewModel {
    private(set) var snapshot: TodaySnapshot = .empty
    private(set) var state: LoadState = .idle

    private let store: MetricsReading
    private let calendar: Calendar

    init(store: MetricsReading, calendar: Calendar = .current) { ... }

    func load() async                // .task
    func refresh() async             // .refreshable
    func select(_ date: Date) async  // WeekStrip binding
}
```

Inside `load()`, in order:

1. Fetch the visible month plus enough history for the streak, most-recent-first.
2. Fetch `GoalTargets`.
3. `evaluate` each day once, into a date-keyed dictionary — **not once per dot**, which is what a view-body implementation ends up doing.
4. Ask `MonthGridLayout` for the cells, mapping `DayStatus` to `DotState`. This mapping lives here because `Persistence` must not depend on `DesignSystem`.
5. Compute `currentStreak`.
6. Publish one `TodaySnapshot`. A single assignment means a single render.

4.5 View

```
struct TodayScreen: View {
    @State private var vm: TodayViewModel
    @Environment(\.colorScheme) private var scheme

    init(store: MetricsReading) {
        _vm = State(initialValue: TodayViewModel(store: store))
    }

    var body: some View {
        ScrollView {
            VStack(alignment: .leading, spacing: 24) {
                TodayHeader(snapshot: vm.snapshot)
                WeekStrip(selection: Binding(
                    get: { vm.snapshot.selectedDate },
                    set: { date in Task { await vm.select(date) } }
                ))
                SolidCard { monthSection }
                TodayTileGrid(snapshot: vm.snapshot)
            }
            .padding(20)
            .padding(.bottom, 100)
        }
        .background(LifeOSTokens.canvas.resolve(scheme))
        .task { await vm.load() }
        .refreshable { await vm.refresh() }
    }
}
```

No `@Query`, no `modelContext`, no arithmetic. Every subview takes the snapshot or a slice of it, which is what makes previews trivial: construct a `TodaySnapshot` literal and render.

4.6 Edge cases and tests

CONDITION	BEHAVIOUR
Month boundaries	Leading blanks from <code>firstWeekday</code> , padded to whole rows — tested in <code>MonthGridLayout</code>
Future days in the current month	Rendered <code>.future</code> by position, never evaluated against goals
No data for a day	Faint outline dot; excluded from the streak
Goal changed in Settings	Next <code>load()</code> re-evaluates all history; dots change
Recovery tile before Whoop is connected	Em dash — never <code>0%</code>

TESTS

- `PersistenceTests` — evaluation and streak arithmetic (pure, terminal, no simulator).

- `DesignSystemTests` – month grid maths against a fixed UTC calendar.
- `LIfе0STests` – `TodayViewModel` with a stub `MetricsReading` : an empty store produces `.empty` ; a stubbed month produces the expected cell count and streak; changing targets between two `load()` calls changes the cells.

5 Body

Teal. Your weight at 96 points and nothing competing with it, a weekly delta beneath, and a fourteen-day bar chart where the days you skipped are visibly gaps.

5.1 Intent

WORKING BACKWARDS · V1 FEATURE

Body is a teal screen with your weight as the hero. Below it, the week's change with direction, and a fourteen-day bar chart where each bar is a weigh-in. Days you didn't step on the scale are gaps, not interpolations.

Where does weight come from?

Your Renpho scale, via Apple Health. Renpho already syncs there, so Life OS never talks to Renpho directly — one less integration, one less thing to break, and no third-party credentials to store.

Why a weekly delta instead of a trend line?

Daily bodyweight is mostly water and noise. A seven-day delta is the shortest window where the signal beats the noise, and it's the number that actually changes behaviour.

What if I weigh myself twice in one day?

The row for that day holds one weight; the most recent write wins. For a daily-granularity app the second weigh-in is the better one anyway.

SCREEN ANATOMY

- `GradientCanvas(hue: .body)` behind everything.
- `HeroNumeral` — weight, one decimal, unit always shown.
- Delta row — signed change over seven days, with direction.
- 14-day bar chart — one bar per weigh-in, gaps preserved.

5.2 Files and folders

```
Features/Body/
├── Model/
│   └── BodySnapshot.swift      hero value, delta, bars
└── ViewModel/
```

	└─ BodyViewModel.swift	fetch 14 days, derive delta
	└─ View/	
	└─ BodyScreen.swift	layout only
	└─ WeightBarChart.swift	bars, gaps, no interpolation

5.3 Model

```
struct BodySnapshot: Equatable, Sendable {
    var current: MetricReading<Double> // kg, nil when never weighed
    var weeklyDeltaKg: Double? // nil when < 7 days of history
    var bars: [WeightBar] // oldest first, gaps included
}

struct WeightBar: Identifiable, Equatable, Sendable {
    var id: Date { date }
    var date: Date
    var weightKg: Double? // nil renders a gap, not a zero bar
}
```

THE GAP IS THE DATA

`WeightBar.weightKg` is optional so the chart can render an absence. Filtering absent days out of the array instead would silently compress the x-axis and make a fortnight of two weigh-ins look like two consecutive days.

5.4 ViewModel

```
@Observable @MainActor
final class BodyViewModel {
    private(set) var snapshot: BodySnapshot = .empty
    private(set) var state: LoadState = .idle
    private let store: MetricsReading

    func load() async
    func refresh() async
}
```

The delta rule, implemented once, here:

- Take the most recent weight as the hero. If there is none, `current.value` stays `nil` and the hero renders an em dash with its unit.
- Find the most recent weight at least seven days older than the hero's date. If there isn't one, **the delta is suppressed entirely** rather than computed over a shorter window — a "weekly change" measured over two days is a wrong number wearing a right label.
- Build fourteen bars by walking the calendar backwards, not by iterating the fetched rows, so missing days become `nil` bars rather than disappearing.

5.5 View

```
struct BodyScreen: View {
  @State private var vm: BodyViewModel

  var body: some View {
    GradientCanvas(hue: .body) {
      ScrollView {
        VStack(alignment: .leading, spacing: 22) {
          HeroNumeral(value: vm.snapshot.current, unit: "kg")
          deltaRow
          GlassCard { WeightBarChart(bars: vm.snapshot.bars) }
        }
        .padding(20)
      }
    }
    .task { await vm.load() }
    .refreshable { await vm.refresh() }
  }
}
```

`WeightBarChart` takes `[WeightBar]` and nothing else — no view model, no store, no dates computed inside it. That is what makes its preview a one-liner and its behaviour on a fortnight of gaps something you can actually look at.

5.6 Edge cases and tests

CONDITION	BEHAVIOUR
Never weighed	Hero is an em dash with unit; delta suppressed; chart renders an empty state
Fewer than 7 days of history	Delta suppressed entirely
Two weigh-ins on one day	Last write wins at the store; the screen sees one value
Weight recorded in pounds	Converted through <code>HKUnit</code> at the boundary; stored and displayed in kg
Gap in the middle of the fortnight	Bar omitted, x-axis position preserved

TESTS

- `BodyViewModel` with a stub store: no data → suppressed delta and nil hero; six days of history → still suppressed; eight days → correct signed delta.
- Bar construction: a stub returning three scattered weigh-ins yields fourteen bars, eleven of them `nil`.

6 Activity

Amber. Today's step count as the hero, three tiles beneath it. It answers "did I move today" before you have finished unlocking the phone.

6.1 Intent

WORKING BACKWARDS · V1 FEATURE

Activity is amber, with today's step count as the hero and three tiles beneath it: active minutes, energy burned, and resting heart rate.

Both my iPhone and Watch count steps. Does that double?

No. Life OS uses `HKStatisticsCollectionQuery`, which de-duplicates overlapping samples across sources. Summing raw samples client-side — the obvious naive approach — would roughly double every step count on days you carried both devices.

Why is resting heart rate here and not on Recovery?

Because it comes from HealthKit and is available in V1, while Recovery's other metrics depend on the Whoop integration landing. It moves to Recovery once that ships — a one-line change to two snapshots, because neither screen computes anything.

6.2 Files and folders

```
Features/Activity/
├── Model/
│   └── ActivitySnapshot.swift
├── ViewModel/
│   └── ActivityViewModel.swift
└── View/
    └── ActivityScreen.swift
```

Three files, because this screen genuinely is hero-plus-tiles with no bespoke component. When a feature needs no private subview, it does not get one.

6.3 Model

```
struct ActivitySnapshot: Equatable, Sendable {
    var steps: MetricReading<Int>
    var activeMinutes: MetricReading<Int>
    var energyKcal: MetricReading<Double>
    var restingHR: MetricReading<Double>
}
```

`steps.goal` is populated from `GoalTargets.steps` so the tile can draw its progress ring. The other three carry no goal and draw none — a progress ring with no target is a decoration pretending to be information.

6.4 ViewModel

```
@Observable @MainActor
final class ActivityViewModel {
    private(set) var snapshot: ActivitySnapshot = .empty
    private(set) var state: LoadState = .idle
    private let store: MetricsReading

    func load() async           // today's row + targets, nothing else
    func refresh() async

}
```

The simplest ViewModel in the app: one row, one targets fetch, one snapshot. It exists anyway, because the value of a uniform structure is that the simple screens look like the complicated ones — and because it is where progress ratios get clamped to `0...1` instead of being clamped inside a view body.

6.5 View

```
GradientCanvas(hue: .activity) {
    VStack(alignment: .leading, spacing: 22) {
        HeroNumeral(value: vm.snapshot.steps, unit: "steps")
        LazyVGrid(columns: [GridItem(.flexible()), GridItem(.flexible())], spacing: 12) {
            MetricTile(label: "Active", reading: vm.snapshot.activeMinutes)
            MetricTile(label: "Energy", reading: vm.snapshot.energyKcal)
            MetricTile(label: "Resting HR", reading: vm.snapshot.restingHR)
        }
    }
}
```

6.6 Edge cases and tests

CONDITION	BEHAVIOUR
Watch on the charger all day	All four readings <code>nil</code> ; four em dashes, no zeros
Steps goal set to a very low value	Progress clamped to <code>0...1</code> ; Settings excludes zero from the range
Very large step count	<code>minimumScaleFactor(0.5)</code> and <code>lineLimit(1)</code> on the hero
iPhone and Watch both counting	De-duplicated upstream in <code>HealthData</code> ; the screen never sees the raw samples

TESTS

- `ActivityViewModel` : an empty store yields a snapshot whose four readings are all `nil` — the regression test for "did someone default a metric to zero."
- Progress clamping at goal boundaries: exactly-at-goal is `1.0` , over-goal stays `1.0` .

7 Recovery

Blue. Whoop's recovery percentage as the hero — the number you would otherwise open the Whoop app to see — with HRV, day strain, and sleep performance beneath it.

7.1 Intent

WORKING BACKWARDS · V1 FEATURE

Recovery leads with Whoop's recovery percentage, with HRV, day strain, and sleep performance beneath it.

Whoop syncs to Apple Health. Why build an OAuth integration at all?

Whoop writes sleep, workouts, and heart rate to Health, but **not Recovery % or Day Strain** — those are proprietary and exist only through Whoop's own API. They are also the two numbers most people wear the strap for.

What does this screen show before Whoop is connected?

A designed empty state — an em dash and "Connect Whoop in Settings" — never `0%`. A zero recovery score is a medically alarming number to display to someone who simply hasn't logged in yet.

What if I didn't wear the strap last night?

Whoop returns no recovery record for that cycle, the field stays `nil`, and the screen shows the same empty state. No interpolation, no carry-forward from yesterday.

7.2 Files and folders

```
Features/Recovery/
├── Model/
│   └── RecoverySnapshot.swift      readings + connection state
├── ViewModel/
│   └── RecoveryViewModel.swift
└── View/
    └── RecoveryScreen.swift        hero, tiles, connect prompt
```

7.3 Model

```
struct RecoverySnapshot: Equatable, Sendable {
    var recoveryPct: MetricReading<Double>
    var hrvMs: MetricReading<Double>
    var dayStrain: MetricReading<Double>
    var sleepPerformancePct: MetricReading<Double>
    var connection: ConnectionStatus          // shared component model
}
```

The distinction this screen has to carry, and the reason `connection` is in the snapshot: **"not connected"** and **"connected but no data last night"** look identical in the data and mean completely different things to you. Both render an em dash; only the first offers a connect action.

```
enum ConnectionStatus: Equatable, Sendable {
    case connected(lastSync: Date?)
    case disconnected
    case needsReauth
}
```

7.4 ViewModel

```
@Observable @MainActor
final class RecoveryViewModel {
    private(set) var snapshot: RecoverySnapshot = .empty
    private(set) var state: LoadState = .idle

    private let store: MetricsReading
    private let connections: ConnectionReading // Keychain-backed, protocol

    func load() async
    func refresh() async
}
```

Two dependencies, both protocols. `ConnectionReading` reports whether Whoop tokens exist and when the last successful sync happened; it is what `WhoopKit` exposes upward. The ViewModel never touches the Keychain and never knows an HTTP call exists.

7.5 View

```
GradientCanvas(hue: .recovery) {
  VStack(alignment: .leading, spacing: 22) {
    HeroNumeral(value: vm.snapshot.recoveryPct, unit: "%")

    if case .disconnected = vm.snapshot.connection {
      ConnectionRow(status: .disconnected, service: .whoop)
    }

    LazyVGrid(columns: ...) {
      MetricTile(label: "HRV",          reading: vm.snapshot.hrvMs)
      MetricTile(label: "Day strain",    reading: vm.snapshot.dayStrain)
      MetricTile(label: "Sleep performance", reading: vm.snapshot.sleepPerformancePct)
    }
  }
}
```

7.6 Edge cases and tests

CONDITION	BEHAVIOUR
Whoop never connected	Em dash hero plus a connect row. Never 0%
Connected, strap not worn	Em dash hero, no connect row, last-sync timestamp still recent
Token expired	.needsReauth — the row says reconnect, and no silent retry loop runs behind it
Whoop cycle boundary ≠ calendar day	Cycles attributed to the day the cycle starts; documented so the mismatch isn't read as a bug

TESTS

- Three `ConnectionStatus` cases against a stub: each produces the expected snapshot and, crucially, none of them produces a zero.
- A day with recovery data present but HRV missing renders three tiles and one em dash.

8 Settings

Neutral. Where the goals that define a filled dot are edited, where every connection reports its real state, and where sync failure is allowed to be visible — because it is the only place in the app where it is allowed to be visible at all.

8.1 Intent

WORKING BACKWARDS · V1 FEATURE

Settings shows connection status for Health, Whoop, and Supabase, the last time each one delivered anything, and the five numbers that decide what a good day is. Change a goal here and the dot grid on Today re-reads itself against the new standard.

Why can I edit the goals at all?

Because the dot grid is the app's central claim, and a claim you can't calibrate is one you stop believing. The defaults are 8,000 steps, 7h sleep, 30 min exercise, 2,500 ml water, 3 of 4 required.

What does Settings do when HealthKit access was denied?

It cannot know. HealthKit read authorization is opaque by design. Settings therefore shows the last time data arrived and links to the Health app's own permission screen, rather than asserting a state it cannot verify.

Where does the quick log live?

Not here — it is a sheet on the root FAB, reachable from any tab, because logging water is the one thing you do repeatedly and it must never be four taps deep.

8.2 Files and folders

```

Features/Settings/
├── Model/
│   ├── SettingsSnapshot.swift    connections + last-sync times
│   └── GoalDraft.swift           editable copy of GoalTargets
├── ViewModel/
│   └── SettingsViewModel.swift    edit, validate, commit
└── View/
    ├── SettingsScreen.swift
    └── GoalEditorSection.swift    five steppers, one per target

Components/View/QuickLogSheet.swift ← water + weight, presented from the FAB
Components/ViewModel/QuickLogViewModel.swift

```

8.3 Model

```
struct SettingsSnapshot: Equatable, Sendable {
    var health:    ConnectionStatus
    var whoop:     ConnectionStatus
    var supabase:  ConnectionStatus
}

struct GoalDraft: Equatable, Sendable {
    var steps: Int           // 1,000 - 30,000
    var sleepMinutes: Int    // 240 - 660
    var exerciseMinutes: Int // 5 - 180
    var waterML: Double      // 500 - 6,000
    var requiredCount: Int   // 1 - 4

    var targets: GoalTargets { ... }
}
```

The draft is a separate type from `GoalTargets` on purpose. Editing writes to the draft; committing validates it into targets. Binding steppers straight to the persisted model would write to the database on every increment and re-evaluate a month of history mid-drag.

8.4 ViewModel

```
@Observable @MainActor
final class SettingsViewModel {
    private(set) var snapshot: SettingsSnapshot = .empty
    var draft: GoalDraft = .defaults           // mutable - bound via @Bindable

    private let store: MetricsReading & MetricsWriting
    private let connections: ConnectionReading

    func load() async
    func commitGoals() async                 // clamps to range, then writes once
    func reconnectWhoop() async
    func openHealthPermissions()
}
```

`draft` is the one mutable property on any `ViewModel` in this app, and it is mutable because SwiftUI's `@Bindable` needs it to be. Everything else stays `private(set)`: the View proposes, the `ViewModel` disposes.

8.5 View

```
struct SettingsScreen: View {
  @State private var vm: SettingsViewModel

  var body: some View {
    @Bindable var vm = vm
    Form {
      Section("Connections") {
        ConnectionRow(status: vm.snapshot.health, service: .health)
        ConnectionRow(status: vm.snapshot.whoop, service: .whoop)
        ConnectionRow(status: vm.snapshot.supabase, service: .supabase)
      }
      GoalEditorSection(draft: $vm.draft)
    }
    .task { await vm.load() }
    .onDisappear { Task { await vm.commitGoals() } }
  }
}
```

QUICK LOG

`QuickLogSheet` is a component rather than a feature because it is presented from the root, over whichever tab you happen to be on. Its view model holds the two draft values and calls `MetricsWriting.logWater / logWeight`, which upsert into today's row. It is the only writing path in V1 that originates from a person rather than a device.

8.6 Edge cases and tests

CONDITION	BEHAVIOUR
Goal set out of range	Clamped at commit; the stepper ranges make it unreachable in the UI anyway
Goal of zero	Excluded from every range — a zero goal makes progress ratios meaningless
HealthKit denied	Undetectable. Shows last-received timestamp and a link out to Health
Sync stalled	Stale last-sync timestamp. Never a blocking error, never an alert
Whoop token cleared	<code>.needsReauth</code> ; the reconnect action restarts the PKCE flow
Quick log while offline	Committed locally; <code>syncedAt</code> stays <code>nil</code> until the next successful push

TESTS

- `SettingsViewModel`: out-of-range drafts clamp; committing writes exactly once.
- `QuickLogViewModel`: logging water twice in a day accumulates into one row rather than creating two.

9 The Coach

One card every morning: what you did well yesterday, what slipped, and one thing to fix today. Built from your actual numbers, on-device, and structurally unable to invent them.

9.1 Intent

WORKING BACKWARDS · V2 FEATURE

Every morning, one card: what you did well yesterday, what slipped, and one thing to fix today. Built from your actual weight, sleep, recovery, steps, calories, and water. Ask it anything — "why is my recovery tanking?" — and it queries your real records to answer rather than guessing.

Does my health data go to a server?

For the coach, no. It runs on Apple's on-device Foundation Models — free, private, offline, no API key. Only food photos leave the device, because the on-device model is text-only and cannot see an image.

How does it avoid making numbers up?

Tool calling. The model doesn't receive your data in a prompt; it calls Swift functions that query `DailyMetrics` and returns what they hand back. A number it had to look up is a number it cannot hallucinate — and the context stays small enough for a 3B on-device model.

What if the on-device model isn't available?

`SystemLanguageModel.availability` is checked at runtime. On a device without Apple Intelligence, or while the model is still downloading, the coach degrades to unavailable rather than silently shipping health data to a cloud provider without asking.

9.2 Files and folders

```
Features/Coach/
├── Model/
│   ├── CoachBrief.swift           the morning card
│   ├── ChatTurn.swift            one message, role + text + state
│   └── CoachAvailability.swift    available · downloading · unsupported
├── ViewModel/
│   └── CoachViewModel.swift       session lifecycle, streaming, tool wiring
└── View/
    └── CoachScreen.swift
```

```
|— BriefCard.swift
|— ChatTranscript.swift
```

LifeOSKit/Sources/Insights/

the tools themselves – reads Persistence only

WHY THE TOOLS LIVE IN THE PACKAGE, NOT THE FEATURE

The functions the model calls are domain logic with a strict dependency rule: `Insights` may read `Persistence` and nothing else. Putting them in `Features/Coach/` would let a future edit reach for a network client mid-conversation. The feature folder holds the conversation; the package holds what can be asked.

9.3 Model

```
struct CoachBrief: Equatable, Sendable {
    var date: Date
    var wentWell: [String]
    var slipped: [String]
    var oneThing: String
}

struct ChatTurn: Identifiable, Equatable, Sendable {
    enum Role { case you, coach }
    var id: UUID
    var role: Role
    var text: String
    var isStreaming: Bool
}

enum CoachAvailability: Equatable, Sendable {
    case available
    case downloading
    case unsupported(reason: String)
}
```

9.4 ViewModel

```
@Observable @MainActor
final class CoachViewModel {
    private(set) var availability: CoachAvailability = .downloading
    private(set) var brief: CoachBrief?
    private(set) var transcript: [ChatTurn] = []
    var input: String = ""

    private let session: InsightSession // wraps LanguageModelSession

    func start() async // checks availability first
    func generateBrief() async
    func send() async // appends, streams, replaces
}
```

Availability is checked before anything else happens, and a failed check is a rendered state rather than a thrown error. The chat streams into the last `ChatTurn` by mutating `text` and flipping `isStreaming` — the View re-renders one row, not the transcript.

9.5 View

```
switch vm.availability {
case .available:
    ScrollView {
        if let brief = vm.brief { BriefCard(brief: brief) }
        ChatTranscript(turns: vm.transcript)
    }
case .downloading:
    CoachUnavailable(title: "Apple Intelligence is still downloading")
case .unsupported(let reason):
    CoachUnavailable(title: reason)
}
```

9.6 Edge cases and tests

CONDITION	BEHAVIOUR
Device lacks Apple Intelligence	Unavailable state with the reason shown. No cloud fallback without asking
Model still downloading	Reported as downloading with the reason surfaced, not a spinner
Tool returns no rows	Model is told "no data for that range" explicitly, so it says so instead of inventing a trend
Guardrail refusal	Surfaced plainly. Never retried with a reworded prompt to evade it
User asks about a future date	Tool returns empty; no speculation

TESTS

- The tools are tested in `InsightsTests` like any other pure function — given a fixture month, each returns exactly the rows it claims to.
- `CoachViewModel` with a stub session: each availability case renders its state; a streamed response mutates one turn rather than appending several.

10 Integrations

Four external systems and one that had to be cut. Each is confined to its own package target, so none of the detail below is visible from any screen — a ViewModel sees only rows in `DailyMetrics` and a connection status.

10.1 Apple HealthKit

The primary source. Read-only in V1 — the app requests no write permissions, which keeps the authorization surface minimal and means Life OS can never corrupt your Health data.

TYPES READ

METRIC	IDENTIFIER	QUERY
Weight	<code>bodyMass</code>	Anchored
Steps	<code>stepCount</code>	Statistics collection
Active energy	<code>activeEnergyBurned</code>	Statistics collection
Exercise minutes	<code>appleExerciseTime</code>	Statistics collection
Resting HR	<code>restingHeartRate</code>	Anchored
HRV	<code>heartRateVariabilitySDNN</code>	Anchored
Water	<code>dietaryWater</code>	Statistics collection
Sleep	<code>sleepAnalysis</code> (category)	Anchored
Workouts	<code>HKWorkoutType</code>	Anchored

FRESHNESS

- `HKAnchoredObjectQuery` with a **persisted anchor per type**, so each sync fetches only what changed since last time.
- `enableBackgroundDelivery` so iOS wakes the app when new samples land.
- `HKStatisticsCollectionQuery` for daily aggregates — this de-duplicates across sources. Summing raw samples yourself double-counts steps on any day you carried both an iPhone and a Watch.

THE AUTHORIZATION TRAP

HealthKit read authorization is **deliberately opaque**. The API will not tell you whether the user denied a type — only whether you have asked. Branching on read authorization status is therefore always a bug. Query anyway, and treat an empty result as "no data," rendering an explicit empty state. Apple designed it this way so apps cannot detect that you are hiding a condition from them.

10.2 Whoop

Base URL `https://api.prod.whoop.com`, API **v2**. v1 is deprecated. Note that v2 identifies sleep by UUID rather than integer ID, and recovery webhooks reference the associated *sleep* UUID, not the cycle ID — a migration footgun worth knowing before you read the payloads.

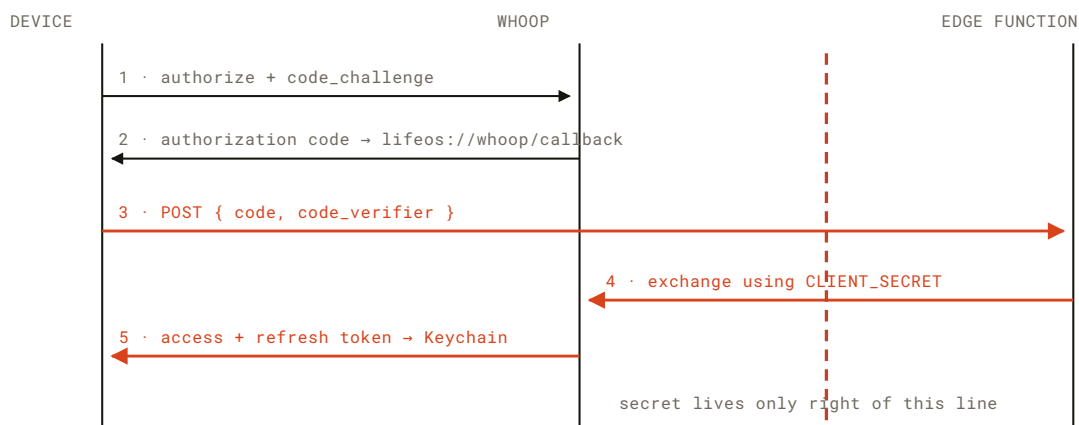
SCOPES

```
offline
read:cycles
read:sleep
read:recovery
read:workout
read:body_measurement
read:profile
```

`offline` is required to receive a refresh token. Without it, the integration breaks the first time the access token expires.

OAuth — AND WHY THE SECRET NEVER SHIPS

Whoop's token exchange requires a client secret. Embedding it in the app binary would make it extractable with `strings`. Supabase Edge Functions solve this cleanly: the app does PKCE and obtains a *code*; the function exchanges it.



Steps 3–5 are the whole point: the client secret is used at step 4, on the server, and never crosses back onto the device. Without this, the integration would be unshippable to anyone but its author.

NIGHTLY JOB AND WEBHOOKS

- A scheduled Edge Function (`pg_cron`) pulls the previous day's cycle, recovery, and sleep overnight, so morning data exists before you open the app.
- A `whoop-webhook` function receives push updates. Signature verified against `WHOOP_WEBHOOK_SECRET`; unverified requests are rejected outright.

- Webhooks are only possible because a public HTTPS endpoint now exists. A phone alone cannot receive them.

10.3 Supabase

SCHEMA

Mirrors the local SwiftData models: `daily_metrics`, `workout_records`, `sleep_records`, `whoop_raw`, and `sync_state`.

`whoop_raw` stores unprocessed API payloads. This is deliberate: when Whoop's response shape differs from the documentation — and it will — you can re-derive every downstream value without re-fetching a month of history against a rate limit.

ROW LEVEL SECURITY

RLS IS ENABLED ON EVERY TABLE, WITHOUT EXCEPTION

Policies scope all access to `auth.uid()`. This is precisely what makes shipping the anon key inside the app safe. An RLS misconfiguration does not throw — it returns an *empty result set*, which looks exactly like "no data yet." Verify policies with a real query before trusting an empty screen.

EDGE FUNCTIONS

FUNCTION	PURPOSE	SECRETS USED
<code>whoop-token-exchange</code>	Exchange auth code / refresh token	<code>WHOOO_CLIENT_SECRET</code>
<code>whoop-sync</code>	Scheduled nightly pull	<code>WHOOO_CLIENT_SECRET</code> , <code>SERVICE_ROLE_KEY</code>
<code>whoop-webhook</code>	Receive Whoop push events	<code>WHOOO_WEBHOOK_SECRET</code> , <code>SERVICE_ROLE_KEY</code>
<code>analyze-meal</code> V3	Food photo → macros via Claude vision	<code>ANTHROPIC_API_KEY</code>

10.4 Foundation Models

The coach runs on Apple's on-device model by default. All three tiers sit behind the same `LanguageModelSession` API, so routing is configuration rather than a rewrite.

TIER	USED FOR	CONTEXT	COST	AVAILABLE
<code>SystemLanguageModel1</code>	Daily brief, chat, trend commentary	Small	Free, private, offline	iOS 26
<code>PrivateCloudComputeLanguageModel1</code>	Monthly / quarterly deep analysis	32K	Free, private	iOS 27
Claude API	Food photo → macros (needs vision)	Large	~cents per photo	Any

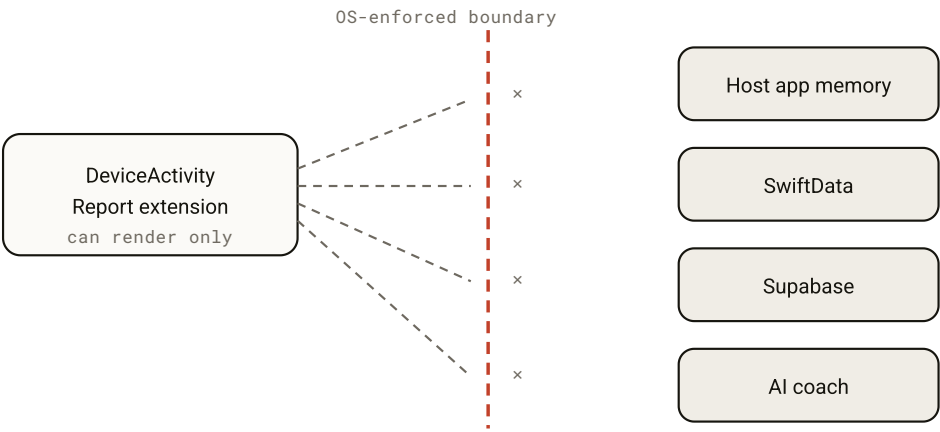
iOS 27 additionally introduces public `LanguageModel1` and `LanguageModelExecutor` protocols, letting third-party providers implement the same interface. **Do not hand-roll a provider abstraction** — Apple ships one, and a bespoke layer would have to be torn out.

TOOL CALLING IS THE ANTI-HALLUCINATION MECHANISM

The model is not handed your data in a prompt. It calls Swift functions that query `DailyMetrics`. Two consequences: a number it had to look up is a number it cannot invent, and the context stays small enough for an on-device model to handle a month of history.

10.5 Screen Time

Screen time was an explicit product requirement. It cannot be built as specified, and the reason is worth documenting precisely so it is not re-attempted.



Usage figures can be displayed inside the report extension and nowhere else. The host app cannot read them into memory, persist them, transmit them, or pass them to the coach.

- **iPhone / iPad:** a screen-time *view* is possible; screen-time *analysis alongside health data* is not. It would also require Apple to grant the Family Controls distribution entitlement.
- **MacBook:** no API exists at any privilege level. Manual entry only, if ever.

Since the entire product thesis is the join across domains, a signal that provably cannot enter the join has no home here. Cut rather than half-built.

11 Data reference

The types every ViewModel in Chapters 4 to 9 reads through. Everything here lives in `LifeOSKit/Sources/Persistence` and depends on nothing.

11.1 DailyMetrics

One row per calendar day. The join key for the entire application.

PROPERTY	TYPE	WRITTEN BY	NOTES
<code>date</code>	<code>Date</code>	—	Always <code>startOfDay</code> . Unique. Never a timestamp.
<code>weightKg</code>	<code>Double?</code>	HealthKit, Quick log	Renpho → Health → here
<code>steps</code>	<code>Int?</code>	HealthKit	De-duplicated across sources
<code>activeEnergyKcal</code>	<code>Double?</code>	HealthKit	
<code>exerciseMinutes</code>	<code>Int?</code>	HealthKit	
<code>sleepMinutes</code>	<code>Int?</code>	HealthKit	Attributed to the wake day
<code>waterML</code>	<code>Double?</code>	Quick log	No automatic source exists
<code>restingHR</code>	<code>Double?</code>	HealthKit	
<code>hrvMs</code>	<code>Double?</code>	HealthKit	SDNN
<code>whoopRecoveryPct</code>	<code>Double?</code>	WhoopKit	Exists nowhere else
<code>whoopDayStrain</code>	<code>Double?</code>	WhoopKit	Exists nowhere else
<code>whoopSleepPerformancePct</code>	<code>Double?</code>	WhoopKit	
<code>updatedAt</code>	<code>Date</code>	MetricsStore	Drives field-level LWW
<code>syncedAt</code>	<code>Date?</code>	SyncKit	<code>nil</code> = never synced

EVERY METRIC IS OPTIONAL, AND THAT IS NOT NEGOTIABLE

A missing value and a zero must never be representable as the same thing. Zero steps means you didn't move; `nil` steps means the watch was on the charger. In a health app a false zero is worse than a blank — it is a wrong number presented with total confidence. This is why `MetricReading` carries an optional all the way to the label.

11.2 UserGoals

Singleton by convention — exactly one row, created with defaults on first access.

PROPERTY	TYPE	DEFAULT	RANGE IN SETTINGS
stepsGoal	Int	8,000	1,000 – 30,000
sleepMinutesGoal	Int	420 (7h)	240 – 660
exerciseMinutesGoal	Int	30	5 – 180
waterMLGoal	Double	2,500	500 – 6,000
requiredCount	Int	3	1 – 4

11.3 Source records

`WorkoutRecord` and `SleepRecord` hold data that is genuinely not daily. They roll *up* into `DailyMetrics`, which means the daily row is always safe to recompute and never the only copy of anything.

11.4 Goal evaluation

```
public func evaluate(_ reading: DayReading,
                    against targets: GoalTargets) -> DayStatus
```

Returns `.onTarget`, `.missed`, or `.noData`.

INPUT	RESULT	RATIONALE
No metrics present at all	<code>.noData</code>	Not a failure; excluded from streaks
$\geq$ <code>requiredCount</code> goals met	<code>.onTarget</code>	Default 3 of 4
Some data, below required count	<code>.missed</code>	An unlogged metric cannot launder a bad day
Exactly at a goal value	Counts as met	<code>>=</code> , not <code>></code>

```
public func currentStreak(statuses: [DayStatus]) -> Int
```

Expects most-recent-first ordering. `.noData` days are transparent — they neither extend nor break a streak. `.missed` terminates immediately.

`DayStatus` is deliberately separate from `DesignSystem.DotState`, because `Persistence` must not depend on `DesignSystem`. The mapping between them happens in a feature ViewModel — see 4.4.

11.5 Store protocols

`MetricsStore` is the only path by which anything writes to `DailyMetrics`. It enforces the one-row-per-day invariant so callers cannot get it wrong. This is what ships today:

```
public struct MetricsStore {
    public init(context: ModelContext, calendar: Calendar = .current)

    @discardableResult
    public func upsert(date: Date, apply: (DailyMetrics) -> Void) throws -> DailyMetrics
    public func upsertBatch(dates: [Date], apply: (Date, DailyMetrics) -> Void) throws
    public func metrics(from: Date, to: Date) throws -> [DailyMetrics]    // most-recent-first
    public func goals() throws -> UserGoals                               // defaults on first ac
}
```

`upsert` normalises any timestamp to `startOfDay` and **merges** — fields the closure doesn't touch are preserved. That is what lets HealthKit and Whoop write the same row safely.

THE PROTOCOLS MVVM NEEDS TO BE ADDED

The concrete API above returns `@Model` instances, which a ViewModel must not hold — see 2.6. Two protocols close that gap, implemented by `MetricsStore` in an extension:

```
public protocol MetricsReading: Sendable {
    func days(from: Date, to: Date) async throws -> [DayRecord]    // most-recent-first
    func targets() async throws -> GoalTargets
}

public protocol MetricsWriting: Sendable {
    func logWater(_ millilitres: Double, on date: Date) async throws
    func logWeight(_ kilograms: Double, on date: Date) async throws
    func updateTargets(_ targets: GoalTargets) async throws
}
```

`DayRecord` is a `Sendable` struct mirroring the columns of `DailyMetrics`. The store converts on the way out.

WHY THE PROTOCOLS RETURN VALUE TYPES

Handing a `@Model` instance to a ViewModel would give every screen a live, mutable handle on the database and make each one require a `ModelContainer` to test. Returning `DayRecord` costs one map per fetch and buys back both.

The protocol methods are `async` while the store's are synchronous. That is deliberate: the store wraps trivially today, and when a fetch eventually moves off the main actor no ViewModel or test has to change.

12 Production edge cases

The register below is the difference between a demo and something you can rely on. Each row is a condition that will occur in normal use, not a theoretical one. The feature chapters cross-reference the rows that apply to them.

12.1 Time and calendars

CONDITION	BEHAVIOUR	STATUS
Sleep spans midnight	Attributed to the <i>wake</i> day via <code>SleepRecord.attributedDate</code>	Designed
Travel across time zones	"Today" follows the device's current zone; a day can be shortened or lengthened	Accepted
DST transition (23h or 25h day)	<code>Calendar.startOfDay</code> handles it; never compute days by adding 86,400 seconds	Designed
Month grid at month boundaries	Leading blanks computed from <code>firstWeekday</code> ; padded to whole rows	Tested
Data arrives for a past day	<code>upsert</code> merges into the existing row; dot grid re-evaluates on next load	Designed
Future-dated day	Rendered <code>.future</code> by position, never evaluated against goals	Tested
Device clock skew vs server	Can cause a stale LWW win	Known limitation

12.2 HealthKit failures

CONDITION	BEHAVIOUR
User denies read access	Undetectable by design. Query anyway; empty result renders an empty state, and Settings links to Health's permission screen
Permission revoked mid-use	Same path — data simply stops arriving; the last-synced timestamp in Settings goes stale
iPhone and Watch both counting steps	De-duplicated by <code>HKStatisticsCollectionQuery</code> . Never sum raw samples
Background delivery doesn't fire	iOS throttles aggressively. Also sync on foreground; never depend solely on background wake
Anchor invalidated (restore from backup)	Anchor reset triggers a full re-read; <code>upsert</code> merge makes this idempotent
Weight recorded in pounds	Read through <code>HKUnit</code> conversion, stored canonically in kg
Watch not worn all day	Fields stay <code>nil</code> . Day is <code>.noData</code> , streak survives
Simulator has no health data	Expected. Seeded data covers development; integration is device-only

12.3 Whoop failures

CONDITION	BEHAVIOUR
Access token expires mid-request	Refresh via Edge Function, retry once, then surface reconnect
Refresh token rotated	Whoop rotates on use — persist the <i>new</i> refresh token atomically. Reusing an old one invalidates the whole chain
Refresh fails with auth error	Clear Keychain, show "reconnect Whoop" in Settings. No silent retry loop
Rate limited (429)	Respect <code>Retry-After</code> ; exponential backoff in the nightly job
Webhook delivered twice	Idempotent by record UUID — reprocessing is a no-op
Webhook signature invalid	Rejected before parsing. Never trust an unverified payload
Strap not worn	No recovery record; fields stay <code>nil</code> ; screen shows its empty state
Whoop cycle boundary $\neq$ calendar day	Cycles are attributed to the day the cycle <i>starts</i> ; documented so the mismatch isn't read as a bug
Payload shape differs from docs	Raw payloads retained in <code>whoop_raw</code> ; re-derive without re-fetching
User revokes access in Whoop's app	Next refresh 401s → same reconnect path

12.4 Sync failures

CONDITION	BEHAVIOUR
No network	Every screen reads local. Failure surfaces only as a stale timestamp in Settings
Write while offline	Committed locally, <code>syncedAt</code> stays <code>nil</code> , pushed on next successful sync
Partial batch failure	Per-row <code>syncedAt</code> means only unsynced rows retry. No all-or-nothing batches
Same row edited locally and by the nightly job	Field-level LWW — the job's Whoop columns and your water column both survive
Supabase session expires	Refresh silently; on hard failure, sync pauses and Settings shows it. The app keeps working
RLS policy misconfigured	Returns empty, does not throw. Indistinguishable from "no data" — verify policies explicitly
Schema drift SwiftData ↔ Postgres	Decoding failures logged per-row and skipped; one bad row never halts a sync

12.5 UI and data display

CONDITION	BEHAVIOUR
First launch, no data	Seeded 60 days in development; in production every hero shows its designed empty state
Metric missing	Em dash at 35% opacity — never <code>0</code> . Implemented once, in <code>MetricValueLabel</code>
Goal changed retroactively	All history re-evaluates on the next <code>load()</code> . Intentional: the grid measures against your current standard
Streak across a no-data gap	Gap is transparent; streak continues
Progress bar with goal of zero	Ratio clamped to 0...1 in the ViewModel; goal ranges in Settings exclude zero
Weight delta with < 7 days history	Delta suppressed entirely rather than computed from a short window
Very large numbers	<code>minimumScaleFactor(0.5)</code> and <code>lineLimit(1)</code> on hero numerals
Dark mode	Every hue has a dark variant; neutral tokens resolve per scheme. No colour defined only inside a scheme branch
Dynamic Type at accessibility sizes	Hero numerals scale down; tiles reflow. Verify at XXL before shipping

12.6 AI failures V2

CONDITION	BEHAVIOUR
Device lacks Apple Intelligence	<code>SystemLanguageModel.availability</code> checked; coach degrades to unavailable rather than silently uploading health data
Model still downloading	Reported as unavailable with the reason surfaced, not a spinner
Context window exceeded	Tool calling keeps context small by construction; deep analysis routes to the 32K PCC tier
Guardrail refusal	Surfaced plainly. Never retried with a reworded prompt to evade it
Tool returns no rows	Model is told "no data for that range" explicitly, so it says so instead of inventing a trend
User asks about a future date	Tool returns empty; no speculation

13 Operations

13.1 Setup runbook

1 · LOCAL BUILD

```
git clone https://github.com/shivvyas2/LifeOS.git
cd LifeOS
cp Config/Secrets.example.xcconfig Config/Secrets.xcconfig
swift test --package-path LifeOSKit      # no simulator needed
open LIfeOS.xcodeproj
```

In Xcode: Swift Language Version → 6, Minimum Deployments → iOS 26.0, and add `LifeOSKit` via *Add Package Dependencies* → *Add Local*. The app target's folders under `LIfeOS/` are picked up automatically – see 2.3.

2 · SUPABASE

```
supabase init
supabase link --project-ref <ref>
supabase db push                # schema + RLS policies
supabase secrets set WHOOP_CLIENT_SECRET=... \
                             WHOOP_WEBHOOK_SECRET=...
supabase functions deploy whoop-token-exchange
supabase functions deploy whoop-sync
supabase functions deploy whoop-webhook
```

Then put the project URL and **anon** key into `Config/Secrets.xcconfig`.

3 · WHOOP DEVELOPER APP

1. Register at `developer.whoop.com` (requires an active Whoop membership).
2. Redirect URI: `lifeos://whoop/callback`
3. Request all seven scopes listed in 10.2, including `offline`.
4. Client ID → `Config/Secrets.xcconfig`. Client secret → `supabase secrets set`, **never the xcconfig**.
5. Webhook URL → the deployed `whoop-webhook` function URL.

4 · DEVICE

HealthKit background delivery and the Whoop OAuth round trip cannot be verified in the simulator. Both require a physical device with a signing team configured.

13.2 Secrets

There is no runtime `.env` on iOS. The app is a signed bundle; anything shipped inside it is extractable. The governing rule is therefore: **what must never leak, must live server-side.**

TIER	LOCATION	CONTENTS	IN GIT?
1 · Build config	<code>Config/Secrets.xcconfig</code>	Supabase URL, anon key, Whoop client ID	Gitignored. Template committed
2 · Runtime	Keychain	Whoop access + refresh tokens, Supabase session	Never
3 · Server	Edge Function env	<code>WHOOP_CLIENT_SECRET</code> , <code>WHOOP_WEBHOOK_SECRET</code> , <code>ANTHROPIC_API_KEY</code> , <code>SERVICE_ROLE_KEY</code>	Never. Never on device either

Nothing in Tier 1 is truly secret. The Supabase anon key is *designed* to ship in clients — it is safe precisely because RLS gates every row — and the Whoop client ID is public by OAuth design. Tier 1 is gitignored for hygiene, not because a leak would be catastrophic.

SERVICE_ROLE IS THE ONE THAT MATTERS

It bypasses RLS entirely. It belongs only in an Edge Function environment — never in an xcconfig, never in the app target, never in git. Every other secret in this system is recoverable by rotation; this one grants total read and write access to everything.

13.3 Testing strategy

Logic that can be tested without a simulator lives in the package and is developed test-first. ViewModels are tested in the app's own test target against stub stores. SwiftUI view bodies are verified by previews and a build — asserting on view structure is ceremony that breaks on every layout change without catching real defects.

TARGET	WHAT IS TESTED	HOW
Persistence	Goal evaluation, streak arithmetic, upsert invariants, rollup derivation	Swift Testing, in-memory <code>ModelContainer</code>
DesignSystem	Month grid calendar maths, hue completeness	Swift Testing, fixed UTC calendar
HealthData	Aggregation and anchor handling	Protocol-wrapped store, fixtures
WhoopKit	v2 payload decoding, token refresh state machine	Recorded payload fixtures
SyncKit	Field-level conflict resolution	Constructed clock skew
Insights V2	Each coach tool against a fixture month	Swift Testing, pure functions
LIfe0STests	Every feature ViewModel: empty store, partial data, state transitions	Stub <code>MetricsReading</code> ; needs a simulator
Views	Both themes, every hue, empty states	SwiftUI previews over snapshot literals
Integrations	Background delivery, OAuth round trip	Manual, physical device

The package declares a macOS platform alongside iOS purely so `swift test --package-path Life0SKit` runs from the terminal without booting a simulator. Integration modules guard platform-specific imports with `#if canImport(HealthKit)` to preserve this.

THE ONE TEST THAT MATTERS MOST

For every screen: an empty store must produce a snapshot whose readings are all `nil` . It is a four-line test per feature and it is the regression guard against the single worst bug this app can have — a confident zero where there is no data.

A Product thesis

A.1 The thesis

Every health app you already have is a silo. Whoop knows your recovery but not that you skipped breakfast. Apple Health knows your steps but not that you blew the grocery budget and slept badly because of it. Each one gives advice that is correct in isolation and useless in context.

Life OS is built on a single bet: **the value is in the join, not the collection**. A coach that can see you missed meditation, logged 5 hours of sleep, journalled "low focus," and went 900 calories over can say something no single-purpose app can. That is the entire reason to build one app instead of using five.

Everything in the architecture follows from that bet. There is one table keyed by date that every source writes into and every reader reads from. Cheap manual modules ship early precisely *because* they make the coach smarter, not despite being simple.

A.2 Product PR FAQ

WORKING BACKWARDS · PRODUCT

Life OS is a personal dashboard that pulls together everything your devices already know about you — weight from your scale, recovery from your strap, movement from your watch, and what you ate from a photo — and answers one question every morning: *am I on track?* Open it and a grid of dots shows the month at a glance. Filled means a good day. The app knows what a good day means because you told it.

Who is this for?

One person. This is a personal app, sideloaded on the author's own devices. There are no accounts, no other users, and no App Store distribution. That constraint removes an enormous amount of architecture and is load-bearing throughout this document.

Why not just use Apple Health?

Apple Health is an excellent store and a poor interpreter. It will show you a chart of anything and tell you the meaning of nothing. Life OS reads from it, then adds the layer Health deliberately doesn't: a definition of a good day, a streak that survives days you forgot to log, and eventually a coach that reads across domains.

What is explicitly *not* in this product?

Screen time analysis. It was requested and it cannot be built — see 10.5 for the technical reason. Rather than ship a hollow version, it is cut.

What's the hardest part?

Not the integrations. It's friction: any app that requires daily manual logging dies in about eleven days. Every product decision here is subordinate to keeping the daily cost near zero — which is why food logging is a photo, not a form.

A.3 Roadmap

V1 is the only slice specified in detail. Each subsequent slice gets its own spec and plan, and adds one feature chapter to this handbook.

SLICE	CONTENTS	WHY HERE
V1	Design system, local data layer, Supabase sync, HealthKit, Whoop, five screens	De-risks the hard integrations before adding breadth
V2	AI coach — daily brief + chat, on-device with tool calling	Needs the data layer to exist first
V3	Nutrition — photo → Claude vision → macros; water logging	The only source that needs a network model
V4	Habits + streaks, Goals + progress bars	Cheap by construction; makes the coach smarter
V5	Journal, Notes	Mood and focus are strong coach signals
V6	Money — manual income, expenses, net worth, savings rate	Manual entry; no bank integration
V7	Content planner calendar	Independent of everything else

WHY THE CHEAP MODULES AREN'T AN AFTERTHOUGHT

V4–V7 are each one SwiftData model, one hue token, and one feature folder assembled from existing components. They are cheap *because* V1 builds the design system, the daily spine, and the MVVM template properly. They also carry disproportionate value: every domain added to the join makes the coach materially better at its job.

B Decision log

DECISION	CHOICE	REASON
Audience	Personal only	Removes accounts, App Store review, multi-user architecture
UI architecture	MVVM, one folder per screen	Derivation becomes testable without a simulator; a file's path names its layer
Views and the database	No <code>@Query</code> in a view	Keeps persistence types out of layout code and derivation out of <code>body</code>
Store surface	Protocols returning value types	Stubbable in tests, and nothing above the store can mutate the database
Shared UI	<code>Components</code> / , split <code>Model</code> / <code>ViewModel</code> / <code>View</code>	A shared piece that owns state has an obvious home
Weight source	HealthKit via Renpho's sync	No Renpho API integration needed at all
Whoop	Direct API in V1	Recovery % and Strain exist nowhere else
Whoop client secret	Edge Function only	Would otherwise be extractable from the binary
Food logging	Photo → AI estimate	Lowest friction; friction is what kills these apps
AI coach	On-device Foundation Models, tool calling	Free, private, offline; can't hallucinate looked-up numbers
Provider abstraction	Use Apple's, don't build one	iOS 27 ships <code>LanguageModel1</code> protocols
Data topology	Local-first, Supabase syncs	A health app that waits on the network is one you stop opening
Conflict resolution	Field-level LWW	Row-level would let the nightly job discard your manual entries
Screen time	Cut	Apple's API forbids reading the values into the app
Every metric optional	Yes	A false zero is worse than a blank in a health app
Missing-value rendering	One component, <code>MetricValueLabel</code>	A rule implemented in nine places is broken in one of them
Dot grid rule	3 of 4, configurable	A vague rule makes the whole grid meaningless
No-data days	Excluded from streaks	Punishing a charging night teaches distrust of the streak
V1 scope	Foundation + health	De-risks the hard integrations before adding breadth

Life OS Engineering Handbook · Revision B · 10 August 2026 · Source of record:

`docs/superpowers/specs/2026-08-10-life-os-design.md` · Page numbers in the contents are regenerated
by `docs/handbook/render-pdf.sh` .